

Projeto de Software e Gestão Ágil

Introdução

Prof. Eduardo Santana

CRUD

Reclamação que ouço quase todo semestre:

"Professor, só fazemos CRUD!"

CRUD

- API Simples (Criar, Listar, Atualizar e Excluir)
- Praticamente sempre a mesma coisa
- Nenhum grande desafio

Exemplos de CRUD

Exemplos de CRUD

- E-commerce
- Banco e Contas
- Rede Social
- Sistema Acadêmico
- Sistema de Reservas
- Sistema de Entregas
- ...

Mas essas empresas são CRUD?



O que diferencia essas empresas?

O que diferencia essas empresas?

- Muitos usuários
- Grande volume de dados
- Número de requisições
- Alta disponibilidade
- Alvo de ataques
- Falhas = perda financeira
- Flutuação na demanda (horários de pico)
- Dependência de serviços externos
- Evoluções constantes
- Múltiplas equipes
- ...

Requisitos Funcionais

- Até agora aprendemos a resolver o que a aplicação deve fazer
- Cadastrar usuário, realizar compra, excluir post, ...
- Isso são os requisitos funcionais
- Na maioria dos casos, são simples de implementar, dificilmente teremos uma implementação ultra complexa (ainda mais com IA)

Onde começa a complicar?

- Quando temos que resolver esses problemas:
- Muitos usuários
- Muitos dados
- Número de requisições
- Alta disponibilidade (24x7)
- Alvo de ataques
- Falhas = perda financeira
- ...

Requisitos de qualidade

- Esses requisitos não definem o que a aplicação deve fazer
- Mas como elas devem fazer e o que devem suportar:
 - A aplicação deve suportar X usuários
 - A aplicação deve suportar X req/s
 - O usuário deve ser autenticado e usar autorização
 - Se um token vazar, o invasor não deve conseguir assumir o controle do usuário
 - A conta de um usuário não pode ficar inconsistente
 - Deploy constante e contínuo

Requisitos de qualidade

- Existem dezenas de requisitos de qualidade, muitas vezes chamados também de requisitos não funcionais
- Os mais usuais:
 - **Confiabilidade**
 - **Escalabilidade**
 - **Segurança**
 - **Manutenabilidade**
 - Observabilidade
 - Usabilidade
 - Rastreabilidade
 - Custo
 - ...

Trade-Off

- E para complicar, nos requisitos de qualidade não existe solução perfeita -> trade-off
- Uma solução pode resolver um problema, mas criar outro problema
- Uma ferramenta pode resolver o problema totalmente, mas ter um custo proibitivo, enquanto outra resolve o problema quase totalmente, mas com um custo muito melhor
- Uma linguagem pode ser melhor que outra, mas ninguém da equipe conhece essa outra linguagem
- O papel do arquiteto é avaliar trade-off, e definir o caminho a seguir

Confiabilidade

- Garantir que a aplicação vai executar e vai retornar o valor correto para aquela transação
- Requisitos derivados:
 - Resiliência
 - Disponibilidade
 - Tolerância a Falhas
 - Recuperabilidade
 - Consistência
 - Observabilidade

Confiabilidade

- E como fazer?
 - Idempotência
 - Retry e Timeout
 - Circuit Breaker
 - Replicação
 - Backup
 - Testes

Escalabilidade

- Garantir desempenho mesmo com a variação na carga da aplicação
- Requisitos Derivados:
 - Desempenho
 - Elasticidade
 - Throughput
 - Latência
 - Capacidade
 - Observabilidade

Escalabilidade

- E como fazer?
 - Cache
 - Filas
 - Load Balancer
 - Sharding
 - Protocolos (exemplo REST x gRPC)
 - ...

Segurança

- Garantir que todo o processo de engenharia de software seja seguro, do desenvolvimento a execução
- Requisitos Derivados:
 - Autenticação
 - Autorização
 - Confidencialidade
 - Integridade
 - Auditabilidade
 - Privacidade
 - Observabilidade
 - Recuperabilidade

Segurança

- E como fazer?
 - Usar protocolos (OAuth2)
 - Tokens seguros
 - Rate Limit
 - Secrets
 - Verificação de dependência

Manutenabilidade

- Garantir que a evolução da aplicação seja fácil
- Requisitos Derivados:
 - Testes
 - Padronização

Manutenabilidade

- E como fazer?
 - Usar padrões de projeto
 - Seguir boas práticas de programação
 - Implementar testes
 - Bom processo de engenharia de software

O que é Arquitetura de
Software?

Algumas Definições

... Definição das questões estruturais que incluem organização total e estrutura de controle global da aplicação; protocolos de comunicação, sincronização e acesso a dados; designação de funcionalidade a elementos de design; distribuição física; composição de elementos de design; escalação e desempenho; e seleção entre as alternativas de design.

An Introduction to Software Architecture, David Garlan e Mary Shaw

Algumas Definições

Software architecture deals with the design and implementation of the high-level structure of the software. It is the result of assembling a certain number of architectural elements in some well-chosen forms to satisfy the major functionality and performance requirements such as scalability and availability. Software architecture deals with abstraction, with decomposition and composition.

Kruchten: The Rational Unified Process

Algumas Definições

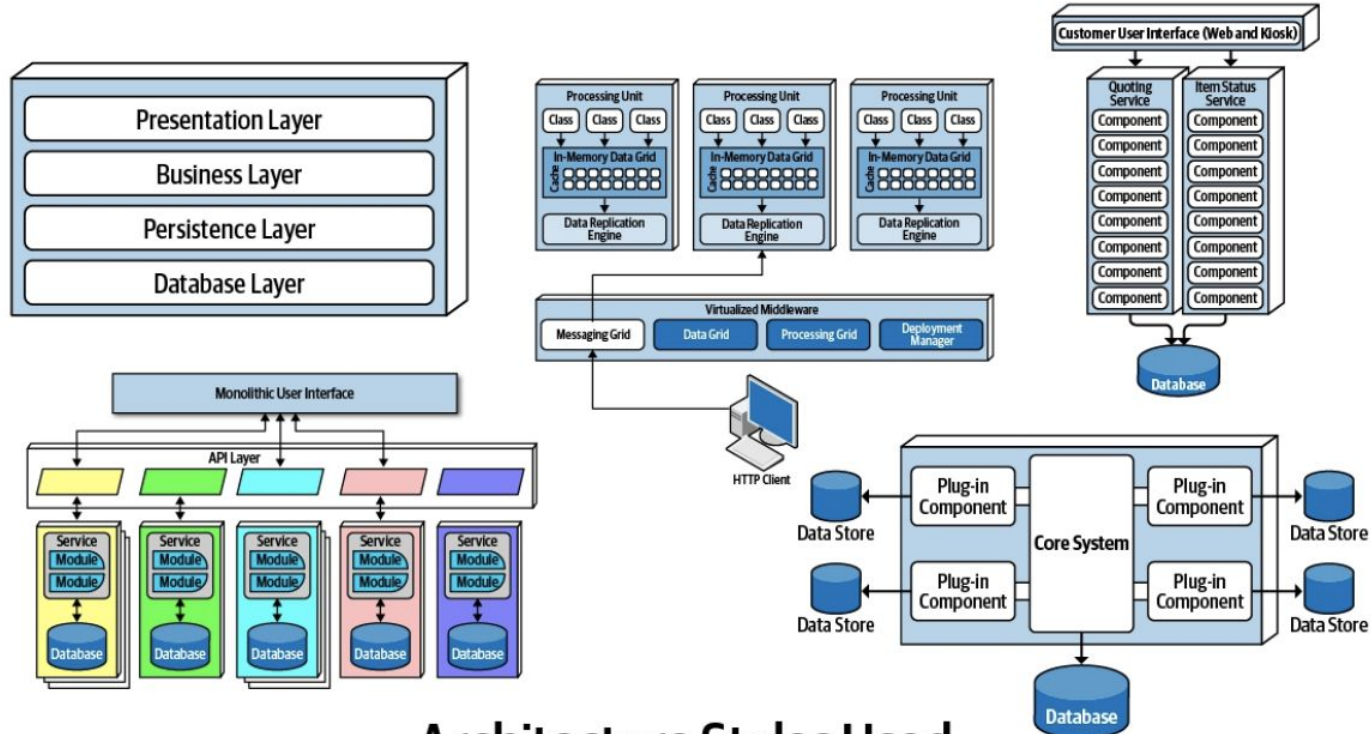
Software architecture consists of the structure of the system, combined with architecture characteristics ("-ilities") the system must support, architecture decisions, and finally design principles.

Fundamentals of Software Architecture

Resumo

Arquitetura de software é tomar decisões estruturais para que um sistema atenda seus requisitos de qualidade, dentro das restrições existentes (custo, tempo de desenvolvimento, ...)

O que é arquitetura de Software?



Architecture Styles Used

Discussão - IA

E a IA nesse contexto?

- IA acelera a engenharia
- Normalmente implementa com menos falhas que humanos

Porém:

- Não é tão boa em avaliar trade-off (pode ajudar, mas a decisão final ainda é humana)
- Não entende todos os riscos envolvidos
- Não consegue entender e prever as consequências de cada decisão (pelo menos por enquanto)

Próxima Aula

- Instalar o Docker Desktop
 - <https://docs.docker.com/desktop/>

Organização da Disciplina

Prof. Eduardo Santana

Objetivos Principais

- Entender a importância da arquitetura de um Projeto de Software
- Entender o papel de um Arquiteto de Software
- Avaliar Trade-Offs em decisões no Projeto de Software

Objetivos Secundários

- Conhecer diferentes modelos arquiteturais
- Conhecer os principais requisitos de qualidade em uma aplicação
- Conhecer diferentes formas de implementar um requisito de qualidade
- Conhecer algumas novas tecnologias (Filas, CI/CD, Docker, OAuth2...)

Organização da Disciplina

- Parte 1:
 - Entender os diferentes modelos arquiteturais
 - Entender o papel de um arquiteto de software
 - Conhecer os principais requisitos não funcionais
- Parte 2:
 - Conhecer os principais requisitos não funcionais relacionados à manutenibilidade de uma aplicação
 - Implementar técnicas e ferramentas para garantir que a manutenibilidade da aplicação
 - Definir processos e ferramentas para a aumentar a confiabilidade em todo o ciclo de desenvolvimento de software

Organização da Disciplina

- Parte 3:

- Conhecer os principais requisitos não funcionais relacionados à confiabilidade de uma aplicação
- Implementar técnicas e ferramentas para aumentar a confiabilidade de uma aplicação
- Definir processos e ferramentas para a aumentar a confiabilidade em todo o ciclo de desenvolvimento de software

- Parte 4:

- Conhecer os principais requisitos relacionados a escalabilidade de uma aplicação
- Implementar técnicas para assegurar que a aplicação suporta a escala necessária sem explodir os custos de infraestrutura
- Definir processo e ferramentas relacionadas a escalabilidade

Organização da Disciplina

- Parte 5:
 - Conhecer os principais requisitos relacionados a segurança
 - Implementar técnicas de autenticação e autorização
 - Definir processo e ferramentas para a segurança em todo o ciclo de desenvolvimento de software

Avaliações

- 2 provas - uma no início e outra no fim do semestre (50% da NF)
 - 21/09 ou 23/09
 - 09/11 ou 11/11
- 1 trabalho desenvolvido durante o semestre (10% da NF) e apresentado ao fim do semestre (40% da NF)

Bibliografia

